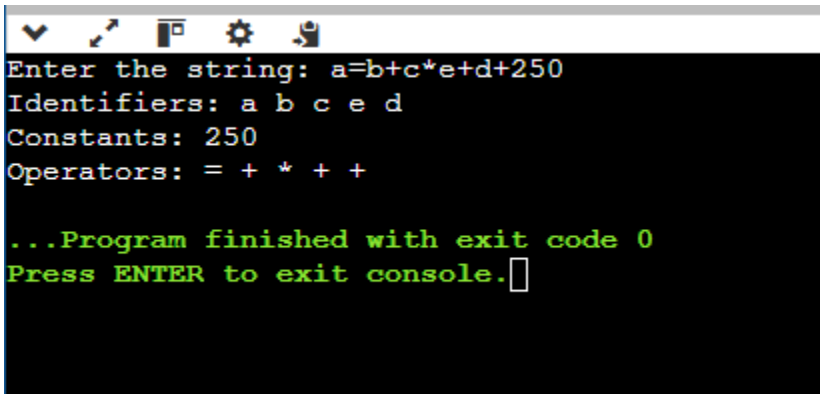
Experiment - 1

Develop a lexical Analyzer to identify identifiers, constants, operators using C program.

CODE :

```
#include <stdio.h>
#include <ctype.h>
int main()
{
   char b[100];
   int i;
   printf("Enter the string: ");
   scanf(" %[^\n]", b);
   printf("Identifiers: ");
   for (i = 0; b[i]; i++)
      if (isalpha(b[i]))
         printf("%c ", b[i]);
   printf("\nConstants: ");
   for (i = 0; b[i]; i++)
      if (isdigit(b[i])) {
         while (isdigit(b[i])) {
            printf("%c", b[i++]);
         }
         printf(" ");
         i--;
      }
   printf("\nOperators: ");
   for (i = 0; b[i]; i++)
      if (b[i] == '+' || b[i] == '-' ||
         b[i] == '*' || b[i] == '=')
         printf("%c ", b[i]);
   return 0;
}
```

OUTPUT :



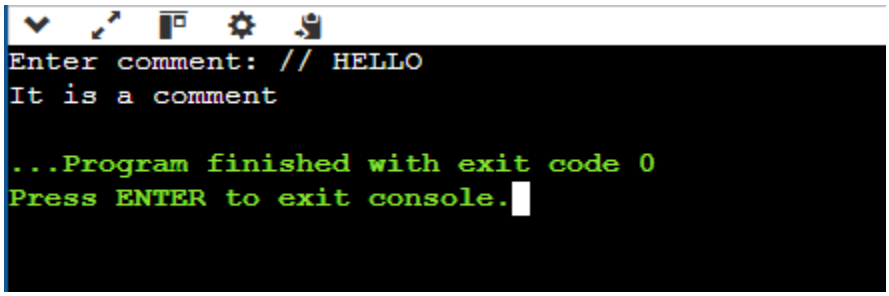
Experiment - 2

Develop a lexical Analyzer to identify whether a given line is a comment or not using C

CODE :

```
#include <stdio.h>
#include <string.h>
int main()
{
   char com[100];
   printf("Enter comment: ");
   fgets(com, sizeof(com), stdin);
   if (com[0] == '/' && com[1] == '/')
      printf("It is a comment");
   else if (com[0] == '/' && com[1] == '*'
         && strstr(com, "*/"))
      printf("It is a comment");
   else
      printf("It is not a comment");
   return 0;
}
```

OUTPUT :


Enter comment: // HELLO
It is a comment
...Program finished with exit code 0
Press ENTER to exit console.

Experiment - 3

Design a lexical Analyzer for given language should ignore the redundant spaces, tabs and new lines and ignore comments using C

CODE :

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>

int main()
{
   char s[200], w[30];
   int i = 0, j;

   printf("Enter the string:\n");
   fgets(s, sizeof(s), stdin);

   while (s[i] != '\0')
   {
      if (isspace(s[i]))
         i++;

      else if (s[i] == '/' && s[i + 1] == '/')
         break;
```

```
        else if (isalpha(s[i]))
        {
            j = 0;
            while (isalnum(s[i]))
                w[j++] = s[i++];
            w[j] = '\0';

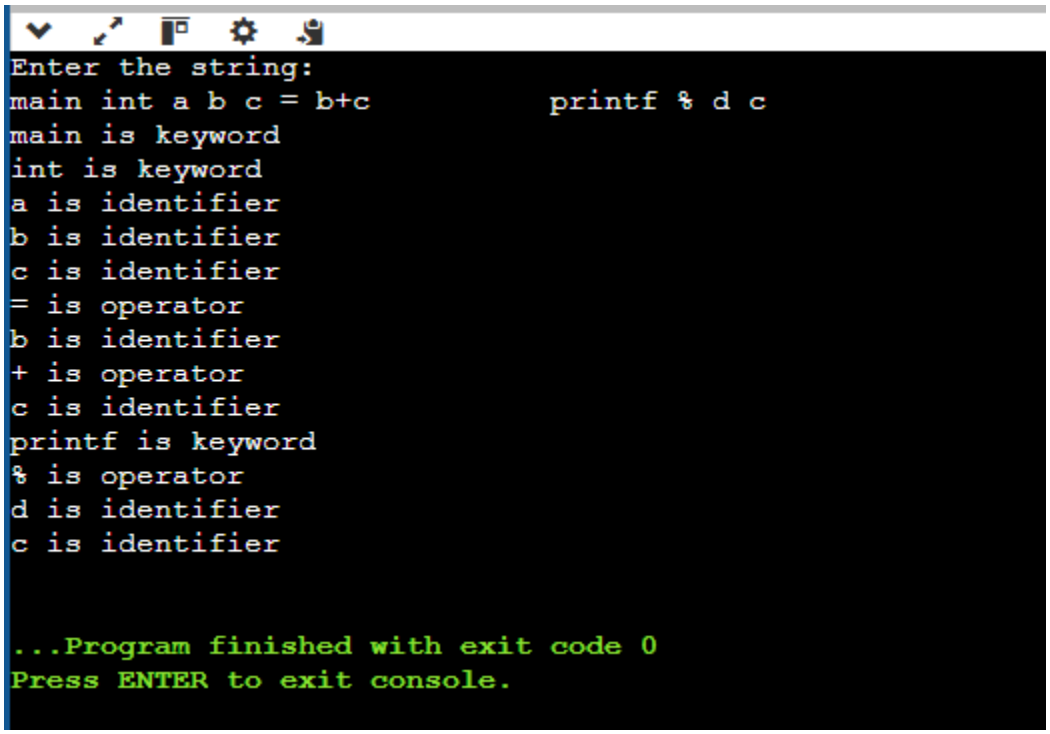
            if (!strcmp(w, "main") || !strcmp(w, "int") ||
                !strcmp(w, "char") || !strcmp(w, "float") ||
                !strcmp(w, "printf"))
                printf("%s is keyword\n", w);
            else
                printf("%s is identifier\n", w);
        }

        else if (s[i] == '=' || s[i] == '+' || s[i] == '-' ||
                 s[i] == '*' || s[i] == '/' || s[i] == '%')
        {
            printf("%c is operator\n", s[i++]);
        }

        else
            i++;
    }

    return 0;
}
```

OUTPUT :



Experiment - 4

Design a lexical Analyzer to validate operators to recognize the operators +,-,*,/ using regular arithmetic operators using C

CODE :

```
#include <stdio.h>
#include <string.h>
```

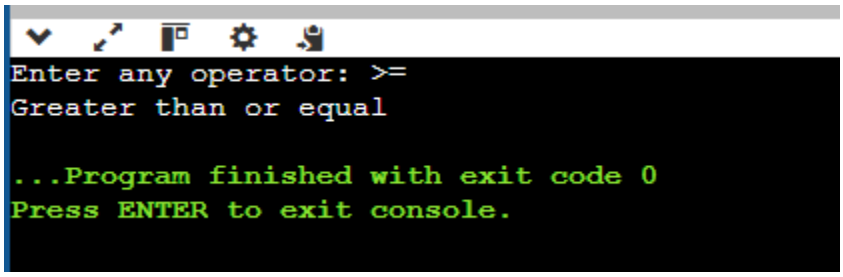
```
int main()
{
   char op[3];

   printf("Enter any operator: ");
   scanf("%s", op);

   if(!strcmp(op,"+"))
      printf("Addition");
   else if(!strcmp(op,"-"))
      printf("Subtraction");
   else if(!strcmp(op,"*"))
      printf("Multiplication");
   else if(!strcmp(op,"/"))
      printf("Division");
   else if(!strcmp(op,"<"))
      printf("Less than");
   else if(!strcmp(op,"<="))
      printf("Less than or equal");
   else if(!strcmp(op,">"))
      printf("Greater than");
   else if(!strcmp(op,">="))
      printf("Greater than or equal");
   else if(!strcmp(op,"=="))
      printf("Equal");
   else if(!strcmp(op,"!="))
      printf("Not equal");
   else
      printf("Invalid operator");
```

```
    return 0;
}
```

OUTPUT :



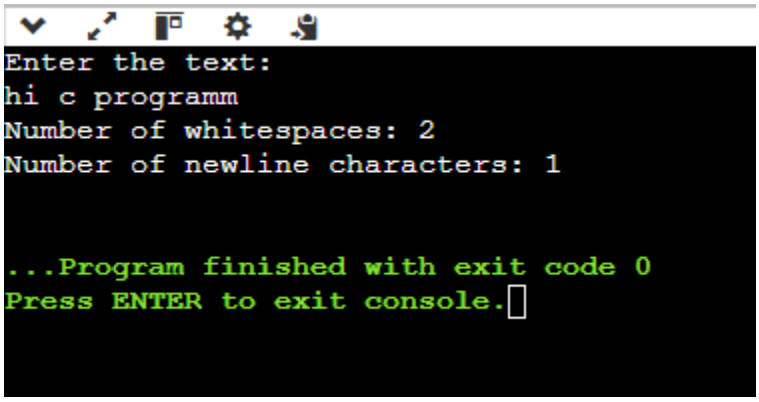
Experiment - 5

Design a lexical Analyzer to find the number of whitespaces and newline characters.

-> CODE :

```
#include <stdio.h>
int main()
{
   char str[200];
   int i, space = 0, newline = 0;
   printf("Enter the text:\n");
   fgets(str, sizeof(str), stdin);
   for(i = 0; str[i] != '\0'; i++)
   {
      if(str[i] == ' ' || str[i] == '\t')
         space++;
      else if(str[i] == '\n')
         newline++;
   }
   printf("Number of whitespaces: %d\n", space);
   printf("Number of newline characters: %d\n", newline);
   return 0;
}
```

OUTPUT :



Experiment - 6

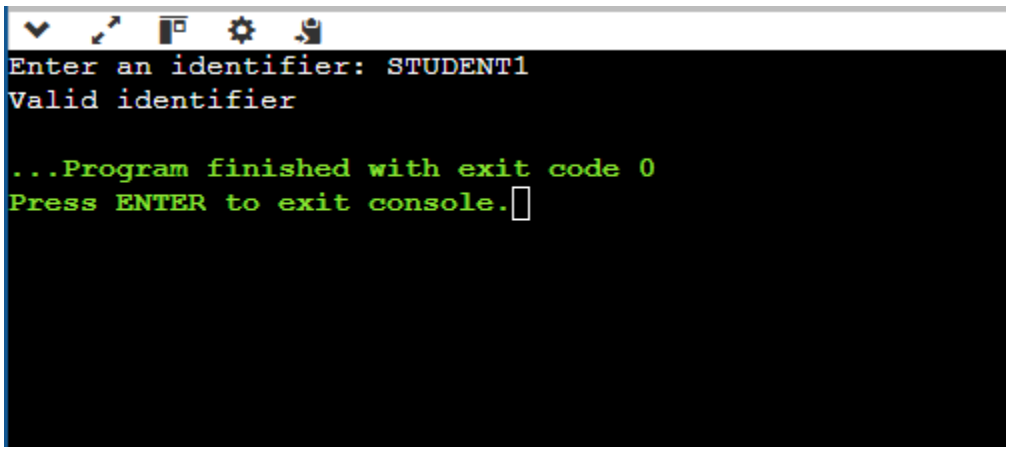
Develop a lexical Analyzer to test whether a given identifier is valid or not.

-> CODE :

```
#include <stdio.h>
#include <ctype.h>
int main()
{
   char id[50];
   int i, valid = 1;
   printf("Enter an identifier: ");
   scanf("%s", id);
   if (!isalpha(id[0]) && id[0] != '_')
      valid = 0;
   for (i = 1; id[i] != '\0'; i++)
```

```
    {
        if (!isalnum(id[i]) && id[i] != '_')
        {
            valid = 0;
            break;
        }
    }
    if (valid)
     printf("Valid identifier");
    else
        printf("Invalid identifier");
    return 0;
}
```

-> OUTPUT :



Experiment - 7

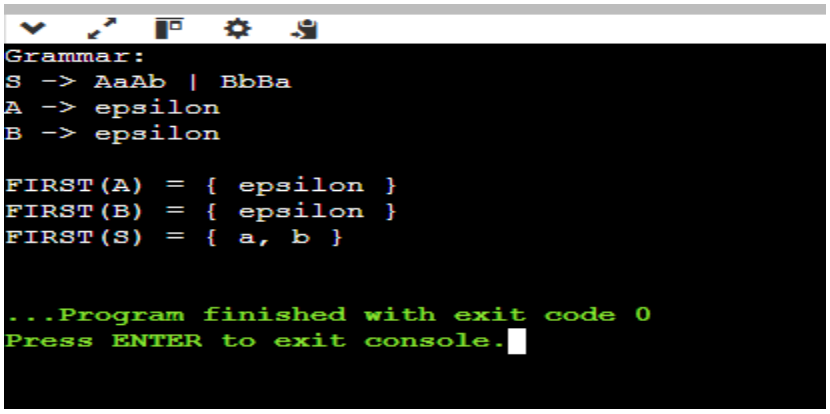
Write a C program to find FIRST( ) - predictive parser for the given grammar
S → AaAb / BbBa
A → ∈
B → ∈

-> CODE :

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>
char prod[3][10] = {"AaAb", "BbBa", "e"};
void first(char c)
{
   int i;
   if (!isupper(c))
   {
      printf("%c ", c);
      return;
   }
   if (c == 'A' || c == 'B')
   {
      printf("e ");
      return;
   }
   if (c == 'S')
   {
```

```
        for (i = 0; i < 2; i++)
        {
            if (prod[i][0] == 'A')
                printf("a ");
            else if (prod[i][0] == 'B')
                printf("b ");
        }
    }
}
int main()
{
    printf("FIRST(A) = { ");
    first('A');
    printf("}\n");
    printf("FIRST(B) = { ");
    first('B');
    printf("}\n");
    printf("FIRST(S) = { ");
    first('S');
    printf("}\n");
    return 0;
}
```

OUTPUT :



Experiment - 8

Write a C program to find FOLLOW( ) - predictive parser for the given grammar
S → AaAb / BbBa
A → ∈
B → ∈

-> CODE :

```
#include <stdio.h>

char grammar[2][5] = {"AaAb", "BbBa"};

void follow(char ch)
{
   int i, j;

   if (ch == 'S')
      printf("$ ");

   for (i = 0; i < 2; i++)
   {
      for (j = 0; grammar[i][j] != '\0'; j++)
      {
         if (grammar[i][j] == ch)
         {
            if (grammar[i][j + 1] != '\0')
               printf("%c ", grammar[i][j + 1]);
            else
               follow('S');
         }
      }
   }
}
```

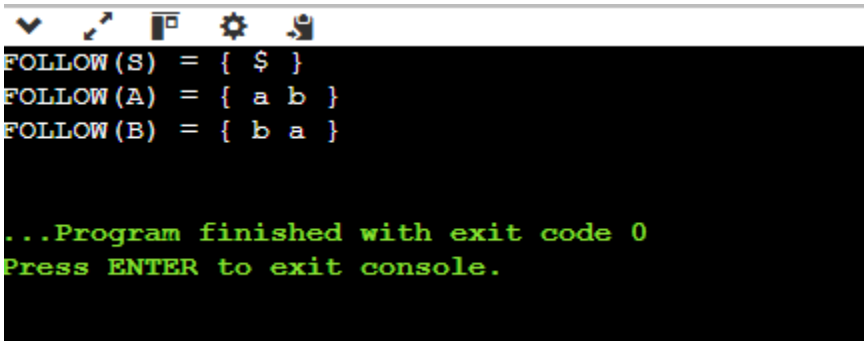
```
int main()
{
   printf("FOLLOW(S) = { ");
   follow('S');
   printf("}\n");

   printf("FOLLOW(A) = { ");
   follow('A');
   printf("}\n");

   printf("FOLLOW(B) = { ");
   follow('B');
   printf("}\n");

   return 0;
}
```

-> OUTPUT :



Experiment 9

Implement a C program to eliminate left recursion from a given CFG.

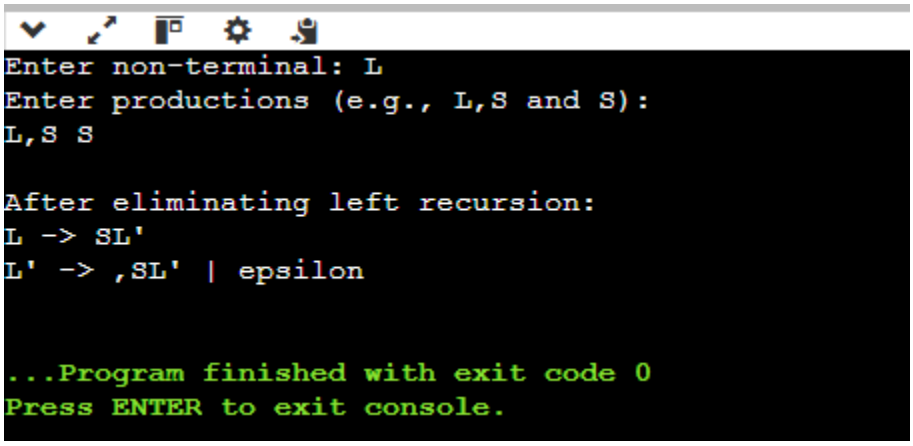
S → (L) / a

L → L , S / S

-> CODE :

```
#include <stdio.h>
#include <string.h>
int main() {
    char A[20], p1[100], p2[100];
    char alpha[100], beta[100];
    printf("Enter non-terminal: ");
    scanf("%19s", A);
    printf("Enter productions (e.g., L,S and S):\n");
    scanf("%99s %99s", p1, p2);
    if (p1[0] == A[0]) {
        strcpy(alpha, p1 + strlen(A));
        strcpy(beta, p2);
        printf("\nAfter eliminating left recursion:\n");
        printf("%s -> %s%s'\n", A, beta, A);
        printf("%s' -> %s%s' | epsilon\n", A, alpha, A);
    } else {
        printf("No immediate left recursion found.\n");
    }
    return 0;
}
```

-> OUTPUT :



Experiment 10

Implement a C program to eliminate left factoring from a given CFG.

S → iEtS / iEtSeS / a

E → b

-> CODE :

```
#include <stdio.h>
#include <string.h>
int main() {
   char A[20], p1[100], p2[100], p3[100], E[100];
   int i = 0;
   printf("Enter non-terminal: ");
   scanf("%19s", A);
   printf("Enter three productions:\n");
   scanf("%99s %99s %99s", p1, p2, p3);
   while (p1[i] == p2[i] && p1[i] != '\0')
      i++;
   if (i > 0) {
      strncpy(E, p1, i);
      E[i] = '\0';
```

```
        printf("\nAfter eliminating left factoring:\n");
        printf("%s -> %s%s'\n", A, E, A);
        printf("%s' -> %s | %s | %s\n",
               A, p1 + i, p2 + i, p3);
    } else {
        printf("No common prefix found.\n");
    }
    return 0;
}
```

-> OUTPUT :

